Manual de instalación: Docker, desarrollo, Render y Android

Proyecto formativo · GPS Guardián Escolar

Equipo del proyecto GPS Guardián Escolar

5 de noviembre de 2026

Versión documental: 2.5.7 · **Estado:** versión editable preparada para subida y revisión de QA/arquitectura.

Tabla de contenido

Guía inicial

Requisitos y comprobaciones previas

Descarga y preparación local

Archivo de variables de infraestructura

Instalación Docker paso a paso

Instalación por componentes

Render y despliegue externo

Android: build, QR y permisos

Desinstalación, conservación y respaldo

Problemas frecuentes

Lista de salida

Referencias y trazabilidad (APA 7.ª edición)

Corte y alcance de evidencia. Revisión documental 2.5.7, 5 de noviembre de 2026. La descripción del **release de referencia** utiliza los commits de main observados en Backend

(1022058d), Frontend (3d9f59fa), DB (9278b1dc) e Infra (131e9895). Se inspeccionaron además las ramas dev/develop más recientes y los suplementos documentales 2.5.6, pero **sus cambios no se consideran integrados a producción por esa sola razón**. La revisión es estática: no se ejecutaron en esta sesión pruebas, despliegues ni validaciones en teléfono. Los resultados operativos requieren evidencia real.

Guía inicial

Dirigida al equipo que instalará el producto en Windows, un servidor Docker o Render. Ejecutar comandos únicamente en el ambiente correcto y con cuentas autorizadas. La instalación requiere ajustes de secretos y servicios; por tanto, los archivos pueden subirse al repositorio, pero **no un archivo .env con valores reales**.

1. Requisitos y comprobaciones previas

Requisito	Uso
Git	Clonación y fijación de versión
Docker Engine + Compose v2	Instalación completa recomendada
Java 17 + Maven Wrapper	Desarrollo del backend
Node.js compatible con Expo 57 + npm	App web/móvil
PostgreSQL 15 + Liquibase (si no usa Compose)	Base de datos
Puertos 5432, 8080, 8081, 8082 según modo	DB, API, web y Metro

Accesos Maps/correo/Expo Push (opcional)

Funciones externas configurables

Las cuatro carpetas deben quedar juntas en una misma carpeta padre porque docker-compose.yaml referencia ../DB-Structure-Guardian-Escolar, ../Backend-Guardian-Escolar y ../Fronted-GuardianEscolar-.

2. Descarga y preparación local

```
mkdir Guardian-Escolar
cd Guardian-Escolar
git clone https://github.com/GPS-Guardian-Escolar/Backend-Guardian-Escolar.git
git clone https://github.com/GPS-Guardian-Escolar/Fronted-GuardianEscolar-.git
git clone https://github.com/GPS-Guardian-Escolar/DB-Structure-Guardian-Escolar.git
git clone https://github.com/GPS-Guardian-Escolar/Docker-Infra-Guardian-Escolar.git
```

Para reproducir el release de referencia, comprobar y fijar los SHA completos registrados en la matriz de control. No hacer git pull indiscriminado de dev sobre main, ni mezclar fuentes de ambientes diferentes.

3. Archivo de variables de infraestructura

```
cd Docker-Infra-Guardian-Escolar
Copy-Item .env.example .env
notepad .env
```

Definir al menos POSTGRES_PASSWORD, APP_DB_PASSWORD (contraseña robusta distinta de la anterior) y JWT_SECRET aleatorio; respetar los requisitos del .env.example del SHA instalado. APP_DB_USERNAME puede utilizar gps_guardian_app; no reutilizar la cuenta de bootstrap para la API. Agregar Maps/correo/Expo solo si el ambiente dispone de proveedores válidos.

Los valores de muestra son nombres de variables, **no credenciales**:

```
POSTGRES_PASSWORD=<secreto_de_bootstrap>
APP_DB_USERNAME=gps_guardian_app
APP_DB_PASSWORD=<secreto_de_aplicacion>
JWT_SECRET=<clave_aleatoria_segura>
LIQUIBASE_CONTEXTS=prod
EXPO_PUBLIC_API_URL=/api
EXPO_PUBLIC_WS_URL=/ws
```

La creación del usuario administrador inicial en un ambiente desplegado requiere contexto de Liquibase aprobado (prod,bootstrap) y variables de identidad/hash; ejecutarlo una vez y retirarlas del entorno después. No convertir contraseñas literales o hashes de prueba en credenciales públicas.

4. Instalación Docker paso a paso

```
# Ubicado en Docker-Infra-Guardian-Escolar
docker compose --env-file .env config --quiet
docker compose --env-file .env up --build -d
```

```
docker compose --env-file .env ps
docker compose --env-file .env logs --tail=100 database liquibase backend frontend
```

Secuencia esperada: PostgreSQL saludable, Liquibase finalizado con código 0, backend saludable y frontend saludable. Esperar a que termine la migración; si falla, leer los logs de DB/Liquibase antes de intentar reinicios. Para verificar:

```
Invoke-WebRequest http://localhost:8080/api/health
Invoke-WebRequest http://localhost:8081/
```

El frontend web se sirve generalmente en http://localhost:8081; el proxy Nginx debe reenviar /api y /ws. Si la salida no es la esperada, no inferir éxito solo porque el navegador abrió la página inicial.

5. Instalación por componentes

Base de datos: provisionar PostgreSQL 15; configurar contraseña/rol; aplicar migraciones del repositorio de DB mediante Liquibase y verificar DATABASECHANGELOG antes de iniciar Spring Boot. **Backend:** desde Backend-Guardian-Escolar, preparar .env desde .env.example, completar DB_URL, DB_USERNAME, DB_PASSWORD, JWT_SECRET y ejecutar ./mvnw spring-boot:run (Linux) o .\mvnw.cmd spring-boot:run (Windows). **Frontend:** desde Fronted-GuardianEscolar-, npm ci, configurar .env con URL de API y ejecutar npm run web:lan o npm run start:lan; el script de Expo Metro usa normalmente el puerto 8082.

Teléfono real: localhost hace referencia al teléfono y no al PC; usar IP LAN autorizada o URL pública HTTPS. Verificar firewall y el mismo Wi-Fi, y nunca publicar el puerto de PostgreSQL hacia Internet para pruebas rápidas.

6. Render y despliegue externo

Crear/configurar PostgreSQL administrado, revisar TLS, accesos y respaldo.

Ejecutar Liquibase de forma separada y controlada contra el destino. El despliegue web de Render **no aplica necesariamente las migraciones**.

Crear servicio backend desde SHA aprobado, variables privadas y puerto adecuado; comprobar health en URL pública.

Crear servicio web Docker frontend; configurar API_UPSTREAM apuntando al backend HTTPS, /api y /ws por proxy, y dominio/certificado válido.

Verificar autenticación, CORS, sesión, Maps, WebSocket, QR e historial con cuentas de ensayo; registrar fecha y SHA, sin exponer tokens.

Una respuesta health UP de DB no garantiza que el proveedor de correo esté configurado ni que el GPS/QR funcionen en dispositivos reales. Evitar publicar resultados no observados.

7. Android: build, QR y permisos

El proyecto usa Expo/EAS y configuración nativa para cámara, GPS, notificaciones y enlaces universales. El comando de build dependerá de los perfiles definidos en eas.json del commit aprobado; revisar primero app.config.js y credenciales de firma. **No se adjunta APK certificado en este paquete**. Validar versión de Android, permisos foreground/background, optimización de

batería, firma y assetlinks.json para que el QR HTTPS abra la app instalada. Documentar la URL del APK únicamente después de publicar y verificar el artefacto.

8. Desinstalación, conservación y respaldo

docker compose --env-file .env down

El comando anterior conserva volúmenes. **No ejecutar docker compose down -v en entornos con datos de valor**, porque elimina el volumen persistente de PostgreSQL. Para migraciones de producción mantener respaldos cifrados y política de recuperación aprobada; no publicar dumps que incluyan datos de estudiantes.

9. Problemas frecuentes

Problema	Posible origen	Revisión recomendada
APP_DB_PASSWORD requerido	Variable omitida	Completar .env sin compartirlo
Liquibase falla	Changelog, credenciales o esquema	logs liquibase y revisar lock/changesets
Backend 500 al consultar rutas	Esquema desactualizado	Migración 014 y columnas/índices
Frontend no accede API		

localhost, proxy o CORS erróneo	
URL, IP y Nginx/headers	
QR no abre Android	
Huella/certificado/deep-link no coincide	
Build EAS y assetlinks.json	
Posición no cambia	
Permisos, ahorro batería, conexión	
recordedAt, logs y estado activo	
Correo no llega	
Proveedor no habilitado o rechazo externo	
Configurar/ensayar SMTP o Brevo	
403 Forbidden	
Rol o estudiante no autorizado	
Revisar permisos; no desactivar seguridad	

10. Lista de salida

La instalación solo se acepta cuando el operador registra SHA, versión de servicios, healthchecks, migración completada, pruebas de acceso, datos ficticios, respaldo restaurable y aprobación QA. Esta guía es instalable y editable, pero no sustituye ese acta.

Referencias y trazabilidad (APA 7.ª edición)

GPS Guardian Escolar. (2026a). *Backend-Guardian-Escolar* [Código fuente, rama main, commit 1022058d]. GitHub. https://github.com/GPS-Guardian-Escolar/Backend-Guardian-Escolar/tree/1022058d545a9e7936edbff75950034ec4f98e88

GPS Guardian Escolar. (2026b). *Fronted-GuardianEscolar-* [Código fuente, rama main, commit 3d9f59fa]. GitHub. https://github.com/GPS-Guardian-Escolar/Fronted-GuardianEscolar-/tree/3d9f59fa3e91b5ecc5b8ae86591187546b966668

GPS Guardian Escolar. (2026c). *DB-Structure-Guardian-Escolar* [Migraciones SQL, rama main, commit 9278b1dc]. GitHub. https://github.com/GPS-Guardian-Escolar/DB-Structure-Guardian-Escolar/tree/9278b1dcb224301536d6158357f74daee74b20ba

GPS Guardian Escolar. (2026d). *Docker-Infra-Guardian-Escolar* [Configuración de despliegue, rama main, commit 131e9895]. GitHub. https://github.com/GPS-Guardian-Escolar/Docker-Infra-Guardian-Escolar/tree/131e98954430e95e9108174eded9896b3f0d5272

GPS Guardian Escolar. (2026e). *Docs-Guardian-Escolar: entregables 2.5.5 y suplementos 2.5.6* [Documentación del proyecto]. GitHub. https://github.com/GPS-Guardian-Escolar/Docs-Guardian-Escolar

Fuentes primarias relevantes: TripController.java, TripService.java, StudentController.java, LocationAvailabilityService.java, WebSocketConfig.java, application.yml, package.json, docker-compose.yaml y 01_ddl/04_alter/014_preferred_student_route.sql, según el documento. Las referencias a rutas son comprobables en los repositorios indicados.